

Extracting data from Facebook's Advertising Platform

SOC6708 ADA

Monica Alexander

Table of contents

1	Overview	2
2	Load in the R packages we need	2
3	Facebook set up	2
3.1	Creating an app through the Facebook developers website	3
3.2	Check version	3
3.3	Obtain app token	3
3.4	Obtain creation act	3
3.5	Set credentials	3
4	Extracting demographic data from Facebook's Marketing API	4
4.1	Add in more demographic characteristics	5
4.2	Exercise	6
5	Extracting more detailed information based on different demographics, behaviors, and interests	6
5.1	Looking up the available demographics	6
5.2	Defining the <code>flexible_spec</code> object	8
5.3	Making the query	8
5.4	More than one group	9
5.5	Exercise	10
6	Extracting population by region	10
6.1	Extract keys for regions	11
6.2	Comparing Facebook populations to Canadian Census	14
6.3	More granular geographies	16

1 Overview

In this module we will be querying Facebook's Marketing API to extract estimates of the number of Facebook users that have certain characteristics, behaviors and interests. Unlike module 1, we will not be relying on an R package which has 'wrapper functions' around API calls, but rather constructing the queries to the API ourselves. The structure of this module is:

- Loading in the R packages we need
- Setting up Facebook API clients
- Extracting basic population estimates
- Extracting population estimates by age and sex
- Extracting more detailed demographic estimates
- Extracting estimates by region

2 Load in the R packages we need

The first step is to load in all packages we need. We are relying on `tidyverse` and `plyr` for data manipulation. The `httr` package allows us to query websites, and the `jsonlite` package helps to manipulate data from JSON files.

```
library(tidyverse)
library(plyr)
library(httr)
library(jsonlite)
library(here)
```

3 Facebook set up

Just like in the first module, in order to make calls to the API, you will need to have a standard Facebook account. If you don't already, you can sign up here: <https://www.facebook.com/>.

3.1 Creating an app through the Facebook developers website

We need to create an app. Go to <https://developers.facebook.com/>. You will need to create a Facebook developer's account. Then, click on My Apps. Then, click Create App, name it and choose Marketing API. You will eventually be led to the app dashboard.

3.2 Check version

Now that we've created an app, we need to get some info to input into our queries. To check the version, select Settings and then Advanced.

Insert the version number into code below and run the code:

```
# insert your version number (it is most likely v18.0)
version <- "vxx.x"
```

3.3 Obtain app token

We also need to get the app token, which is unique to your app. From the menu go to Tools then Graph API Explorer. Copy the User Access Token and paste it where the x's are below and run the code below. Note you will have to play around with permissions (it will need access to ads management and ads read).

```
token <- "xxxxxxxxxxxxxxxxxxxxx"
```

3.4 Obtain creation act

The final piece of information required is your unique creation act. The easiest way to get this info is to go to facebook.com/adsmanager. Then, the url will automatically update and the act number is in the URL. Copy this and paste it below where the x's are and run the code:

```
act <- "xxxxxxxxxxxxxxxxxxxxx"
```

3.5 Set credentials

We now want to put all our credentials (act number, version, and token) into the following string which will allow us to make calls to Facebook's Marketing API.

```
Credentials <- paste0('https://graph.facebook.com/',version,
                      '/act_',act,'/delivery_estimate?access_token=',token,
                      '&include_headers=false&method=get&
                      optimization_goal=REACH&pretty=0&suppress_http_code=1')
```

4 Extracting demographic data from Facebook's Marketing API

Now we've set up our credentials, let's get some data! Firstly, let's extract the number of people who live in Canada on Facebook based on estimates from the Marketing platform. To specify a country, we use the 2-digit ISO code. (You can find a full list on [Wikipedia](#)). Note that it has to be a character with additional quotation marks specified:

```
country <- '"CA"' # Country code
```

We need to specify our query to the API. This starts as defining a string based on our parameters of interest.

```
query <- paste0(Credentials,'&
                targeting_spec={
                  "geo_locations":{"countries":['',country,']},
                  "facebook_positions":["feed","instant_article",
                  "instream_video","marketplace"],
                  "device_platforms":["mobile","desktop"],
                  "publisher_platforms":["facebook","messenger"],
                  "messenger_positions":["messenger_home"]}')
```

Writing the query like this is easier to see but introduces a series of new lines in the string that we don't want. So let's remove those before we make the query:

```
query <- str_replace_all(query, "[\n]", "") |>
        str_replace_all(" ", "")
```

Now we can extract the information using the `url` function. In the following code, that output gets reformatted using the `fromJSON` function. Important note: we have to use the `tidyverse` version of the pipe here (`%>%`) for the code to work.

```
query_val <- url(query) %>% fromJSON
```

If we have a look at the data contained in this object, we can see that there's an estimate of the number of daily active users (dau) in Canada, with a lower and upper bound of the monthly users:

```
query_val$data
```

```
      daily_outcomes_curve estimate_dau estimate_mau_lower_bound
1           0, 0, 0, 0      20636322           22400000
      estimate_mau_upper_bound estimate_ready
1           26400000           TRUE
```

4.1 Add in more demographic characteristics

As well as country of location, we can specify age ranges and gender of users. For example, let's look at the estimated users who are men aged 25 to 55. First, set some variables based on our search parameters. Note that gender = 1 is male and gender = 2 is female. More info on the demographic parameters is [here](#).

```
age_min <- 25
age_max <- 55
gender <- 1
```

Now let's alter our query. The additional conditions of age and gender get specified in the `targeting_spec` JSON in the query, along with `geo_locations`.

```
query <- paste0(Credentials, '&
      targeting_spec={
        "age_min": ', age_min, ',
        "age_max": ', age_max, ',
        "genders": [', gender, '],
        "geo_locations": {"countries": [', country, ']},
        "facebook_positions": ["feed", "instant_article",
        "instream_video", "marketplace"],
        "device_platforms": ["mobile", "desktop"],
        "publisher_platforms": ["facebook", "messenger"],
        "messenger_positions": ["messenger_home"]}')
```

Again we want to remove the new lines:

```
query <- str_replace_all(query, "[\\n]", "") |>
  str_replace_all(" ", "")
```

And extract the info:

```
query_val <- url(query) %>% fromJSON
query_val$data
```

```
      daily_outcomes_curve estimate_dau estimate_mau_lower_bound
1           0, 0, 0, 0           6214882           6800000
      estimate_mau_upper_bound estimate_ready
1           8000000           TRUE
```

4.2 Exercise

Change the parameters in the code in order to retrieve the number of women between 20 and 55 years old that live in Australia that are Facebook users.

5 Extracting more detailed information based on different demographics, behaviors, and interests

In addition to basic demographic characteristics, you can also extract information based on other demographics, behaviors, or interests. The documentation [here](#) provides further information.

5.1 Looking up the available demographics

Broadly, there are three different categories: demographics, interests, and behaviors. The first step in extracting more detailed data is to find out all the possible variables we can choose from. To do this, we are going to query information about our version of the app, using the GET function from the `httr` package.

```
demographics_df <- GET(
  "https://graph.facebook.com/v25.0/search",
  query=list(
    type='adTargetingCategory',
    class='demographics',
    access_token=token,
```

```

    limit=2000
)) %>%
content(as="text", encoding = "UTF-8") %>%
fromJSON %>%
.[[1]]

```

If we take a look at the top of this data, we can see that there's all sorts of interesting information, based on birthdays, family or life events, etc.

```
head(demographics_df)
```

	id	name	type
1	6002714398172	Newlywed (1 year)	life_events
2	6002714398372	Parents (All)	family_statuses
3	6002714398772	Newly-engaged (6 months)	life_events
4	6002737124172	Upcoming birthday	life_events
5	6003054185372	Recently moved	life_events
6	6003053984972	Long-distance relationship	life_events

	path
1	Newlywed (1 year)
2	Parents (All)
3	Newly-engaged (6 months)
4	Birthday, Upcoming birthday
5	Recently moved
6	Long-distance relationship

	description
1	People who have been married for less than 1 year
2	People who are likely to be parents
3	People who have been engaged for less than 6 months
4	People who are going to have their birthday within one week
5	People who have updated their profile with a new current city in the last 6 months
6	People who are in a long-distance relationship

	real_time_cluster	audience_size_lower_bound	audience_size_upper_bound
1	FALSE	15744118	18515083
2	FALSE	348105113	409371614
3	FALSE	2763368	3249721
4	TRUE	63052598	74149856
5	FALSE	764605	899176
6	FALSE	11167000	13132393

In order to include these variables in query, we need to pull out three pieces of information:

the ID, type, and name. Let's get an estimate for people aged 30-40 in Canada who have recently moved. As before, we can set our age and location variables:

```
age_min <- 30
age_max <- 40
country <- "CA"
```

We also need to pull out the information on recently moved:

```
recently_moved_info <- demographics_df |>
  filter(name == "Recently moved")
```

5.2 Defining the flexible_spec object

Now the additional information is defined in a `flexible_spec` JSON object:

```
rm_json <- paste0(',"flexible_spec": [{', recently_moved_info$type,
  '": [{', recently_moved_info$id,
  '","name": "', recently_moved_info$name,
  '"}]}]')
```

5.3 Making the query

Now we want to add this into our query:

```
query <- paste0(Credentials, '&
    targeting_spec={"age_min":', age_min, ',
    "age_max":', age_max,
    "rm_json",' ,
    "geo_locations":{"countries":[' , country, ']}},
    "facebook_positions":["feed", "instant_article",
    "instream_video", "marketplace"],
    "device_platforms":["mobile", "desktop"],
    "publisher_platforms":["facebook", "messenger"],
    "messenger_positions":["messenger_home"]}')
```

```
# remove spaces
query <- str_replace_all(query, "[\\n]", "") |>
  str_replace_all(" ", "")
```



```
# extract
query_val <- url(query) %>% fromJSON
query_val$data
```

```
daily_outcomes_curve estimate_dau estimate_mau_lower_bound
1          0, 0, 0, 0          3956          4300
estimate_mau_upper_bound estimate_ready
1          5100          TRUE
```

5.4 More than one group

Above we were just considering one age group. Let's extract information on the people who recently moved across multiple age groups and plot. Firstly, define the age groups:

```
age_mins <- seq(15, 50, by = 5)
age_maxs <- seq(20, 55, by = 5)
```

Now we can repeat the code above, but loop through the age groups and store in a tibble:

```
recently_moved <- tibble()

for(i in 1:length(age_mins)){
  query <- paste0(Credentials, '&
    targeting_spec={"age_min":', age_mins[i], ',
    "age_max":', age_maxs[i],
    "rm_json":',
    "geo_locations":{"countries":[' , country, ' ]},
    "facebook_positions":["feed", "instant_article",
    "instream_video", "marketplace"],
    "device_platforms":["mobile", "desktop"],
    "publisher_platforms":["facebook", "messenger"],
    "messenger_positions":["messenger_home"]}'}')

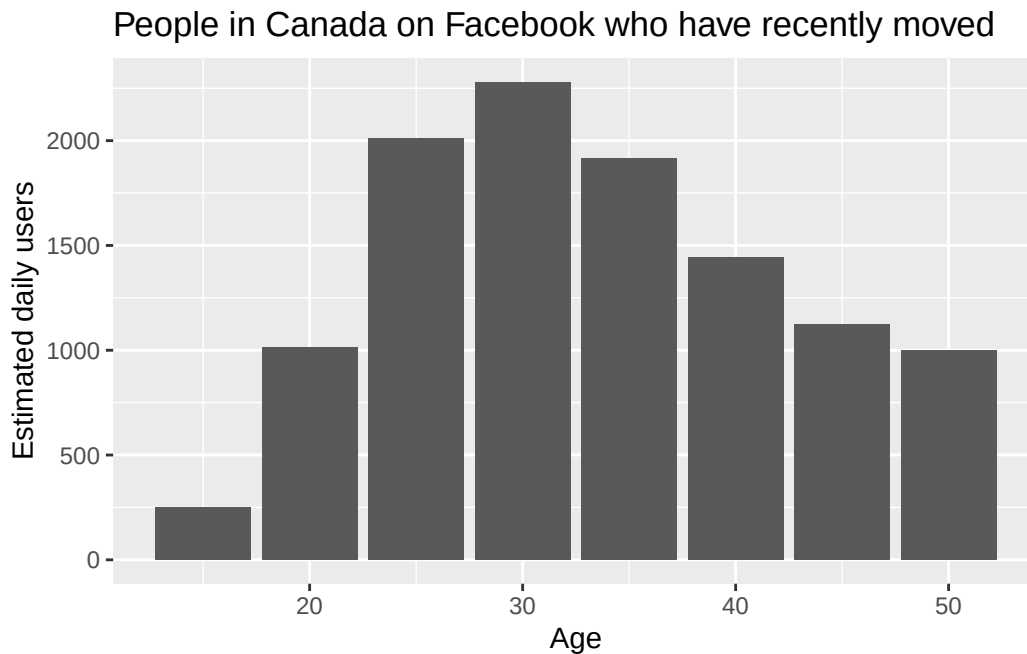
# remove spaces
query <- str_replace_all(query, "[\\n]", "") |>
  str_replace_all(" ", "")

# extract
query_val <- url(query) %>% fromJSON
recently_moved <- bind_rows(recently_moved, query_val$data |>
```

```
    mutate(age_group = age_mins[i]))  
}
```

Now we can plot the estimated users by age group:

```
recently_moved |>  
  ggplot(aes(age_group, estimate_dau)) +  
  geom_bar(stat = "identity") +  
  labs(title = "People in Canada on Facebook who have recently moved",  
       y = "Estimated daily users", x = "Age")
```



5.5 Exercise

Find an estimate of the number of 30-50 year olds in Canada who work in Sales.

6 Extracting population by region

As a final illustration, we are going to extract estimates for the number of Facebook users by Canadian province. Firstly, I'm going to list all the names of the provinces and territories:

```
# Names of provinces and territories
prov_terr <- list(
  "Alberta",
  "British Columbia",
  "Manitoba",
  "New Brunswick",
  "Newfoundland",
  "Northwest Territories",
  "Nova Scotia",
  "Nunavut",
  "Ontario",
  "Prince Edward Island",
  "Quebec",
  "Saskatchewan",
  "Yukon"
)
```

6.1 Extract keys for regions

Before we query the Marketing API, we need to get the unique ‘keys’ for each province/territory. To do this, we are going to query the API meta data, in a similar way to what we did above. For example, to extract information about Alberta:

```
GET(
  "https://graph.facebook.com/v25.0/search",
  query = list(
    location_types = "region",
    type = "adgeolocation",
    q = prov_terr[1],
    access_token = token,
    limit = 3
  )
) %>%
  content(as="text", encoding = "UTF-8") %>%
  fromJSON %>%
  .[[1]]
```

	key	name	type	country_code	country_name	supports_region	supports_city
1	527	Alberta	region	CA	Canada	TRUE	TRUE

Let's make use of dplyr's `ldply` function to get this information in a neat way for all provinces:

```
id_df <- ldply(prov_terr, function(x) {
  GET(
    "https://graph.facebook.com/v25.0/search",
    query=list(
      location_types = "region",
      type = "adgeolocation",
      q = x, # use province name here
      access_token = token,
      limit = 3
    )
  ) %>%
  content(as="text", encoding = "UTF-8") %>%
  fromJSON %>%
  .[[1]]
})
```

Now the `id_df` data frame gives us the required information about the region keys for each province/territory.

`id_df`

	key	name	type	country_code	country_name
1	527	Alberta	region	CA	Canada
2	528	British Columbia	region	CA	Canada
3	529	Manitoba	region	CA	Canada
4	530	New Brunswick	region	CA	Canada
5	531	Newfoundland and Labrador	region	CA	Canada
6	538	Northwest Territories	region	CA	Canada
7	532	Nova Scotia	region	CA	Canada
8	539	Nunavut	region	CA	Canada
9	533	Ontario	region	CA	Canada
10	534	Prince Edward Island	region	CA	Canada
11	535	Quebec	region	CA	Canada
12	536	Saskatchewan	region	CA	Canada
13	537	Yukon	region	CA	Canada
	supports_region	supports_city			
1	TRUE	TRUE			
2	TRUE	TRUE			
3	TRUE	TRUE			

4	TRUE	TRUE
5	TRUE	TRUE
6	TRUE	TRUE
7	TRUE	TRUE
8	TRUE	TRUE
9	TRUE	TRUE
10	TRUE	TRUE
11	TRUE	TRUE
12	TRUE	TRUE
13	TRUE	TRUE

We can use this information to alter our query. As an example, let's extract information for all users in Alberta. Firstly define the key

```
region_id <- id_df[1, "key"]
```

Now instead of having the country in the `geo_locations`, we have region:

```
query <- paste0(Credentials, '&
    targeting_spec={
      "geo_locations":{"regions":[ {"key":', region_id, ' "}]},
      "facebook_positions":["feed","instant_article",
      "instream_video","marketplace"],
      "device_platforms":["mobile","desktop"],
      "publisher_platforms":["facebook","messenger"],
      "messenger_positions":["messenger_home"]}')

# remove spaces
query <- str_replace_all(query, "[\\n]", "") |>
  str_replace_all(" ", "")

# extract
query_val <- url(query) %>% fromJSON
query_val$data
```

	daily_outcomes_curve	estimate_dau	estimate_mau_lower_bound
1	0, 0, 0, 0	2359561	2700000

	estimate_mau_upper_bound	estimate_ready
1	3200000	TRUE

Let's loop over all regions, extract the data, add a column for region name, and save the data:

```

region_estimates <- tibble()
for(i in 1:nrow(id_df)){
  region_id <- id_df[i, "key"]
  query <- paste0(Credentials, '&
    targeting_spec={
      "geo_locations":{"regions":[ {"key":',
      region_id, ' "}]},
      "facebook_positions":["feed","instant_article",
      "instream_video","marketplace"],
      "device_platforms":["mobile","desktop"],
      "publisher_platforms":["facebook","messenger"],
      "messenger_positions":["messenger_home"]}')

  # remove spaces
  query <- str_replace_all(query, "[\\n]", "") |>
    str_replace_all(" ", "")

  # extract
  query_val <- url(query) %>% fromJSON
  this_region <- query_val$data
  this_region <- this_region |>
    mutate(name = id_df[i, "name"])
  region_estimates <- bind_rows(region_estimates, this_region)
  Sys.sleep(3)
}

region_estimates <- region_estimates |>
  select(name, estimate_dau:estimate_mau_upper_bound) |>
  mutate(date_collected = Sys.Date())

write_rds(region_estimates, "../data/fb_ca_province.rds")

```

6.2 Comparing Facebook populations to Canadian Census

It would be interesting to compare the Facebook population distribution to that from the Canadian census. First, let's load in population estimates from StatCan. These are for the last quarter in 2022. I downloaded the data from [here](#).

```

can_pop <- read_csv("../data/canadian_population.csv")

```

Now we want to compute the proportion of the total Canadian population by province/territory:

```
can_pop <- can_pop |>
  mutate(prop = population/sum(population))
```

Let's load in the Facebook data we collected and similarly calculate the proportion of users by province:

```
fb <- read_rds("../data/fb_ca_province.rds")
can_fb_province <- fb |>
  select(name, estimate_dau) |>
  mutate(prop = estimate_dau/sum(estimate_dau)) |>
  dplyr::rename(province = name, n_fb= estimate_dau, prop_fb = prop)
```

Join the proportions together and have a look.

```
can_pop |>
  left_join(can_fb_province) |>
  select(-population, -n_fb) |>
  mutate_at(.vars = vars(prop:prop_fb), funs(round(., 2))) |>
  arrange(-prop)
```

```
# A tibble: 13 x 3
  province                prop prop_fb
  <chr>                   <dbl>  <dbl>
1 Ontario                 0.39    0.37
2 Quebec                  0.22    0.24
3 British Columbia       0.14    0.13
4 Alberta                 0.12    0.12
5 Manitoba                0.04    0.04
6 Nova Scotia             0.03    0.03
7 Saskatchewan            0.03    0.03
8 New Brunswick           0.02    0.02
9 Newfoundland and Labrador 0.01    0.02
10 Prince Edward Island    0        0
11 Yukon                   0        0
12 Northwest Territories    0        0
13 Nunavut                 0        0
```

We can also plot these proportions to better visualize differences in proportions. First, some tidyverse to get the data into a easy plotting format:

```

props_long <- can_pop |>
  left_join(can_fb_province) |>
  select(-population, -n_fb) |>
  mutate(province = fct_reorder(province, prop)) |>
  pivot_longer(-province, names_to = "source", values_to = "prop") |>
  mutate(source = ifelse(source == "prop",
                        "StatCan", "Facebook"))

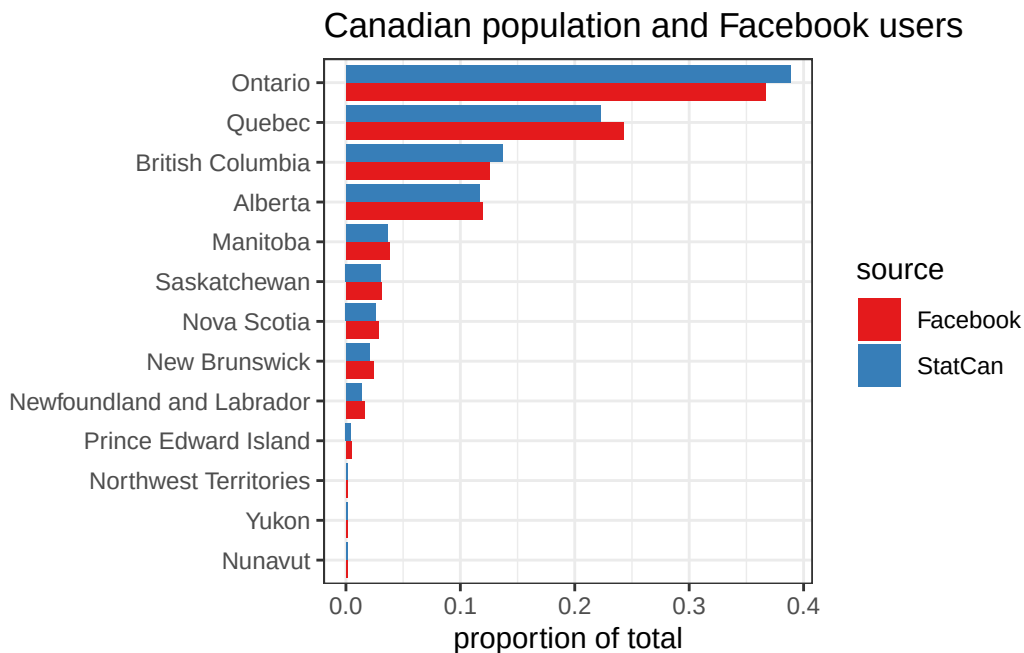
```

Now plot:

```

ggplot(props_long, aes(province, prop, fill = source)) +
  geom_bar(stat = "identity", position = "dodge") +
  coord_flip() +
  labs(title = "Canadian population and Facebook users",
       x = NULL,
       y = "proportion of total") +
  theme_bw() +
  scale_fill_brewer(palette = "Set1")

```



6.3 More granular geographies

We can go even more granular with the geographies, from cities, to neighbourhoods, to zip codes. Details on levels and definitions are given here: <https://developers.facebook.com/docs/graph-api/understanding-graph-ids/>

book.com/docs/marketing-api/audiences/reference/targeting-search/.

For example, let's get the key and estimate of users in Toronto:

```
GET(
  "https://graph.facebook.com/v25.0/search",
  query = list(
    location_types = "city",
    type = "adgeolocation",
    q = "Toronto",
    access_token = token,
    limit = 3
  )
) %>%
  content(as="text", encoding = "UTF-8") %>%
  fromJSON %>%
  .[[1]]
```

	key	name	type	country_code	country_name	region	region_id
1	296875	Toronto	city	CA	Canada	Ontario	533
2	2502315	Toronto	city	US	United States	Ohio	3878
3	2447984	Toronto	city	US	United States	Kansas	3859

	supports_region	supports_city
1	TRUE	TRUE
2	TRUE	TRUE
3	TRUE	TRUE

```
query <- paste0(Credentials, '&
  targeting_spec={
    "geo_locations":{"cities": [ {"key":"' , "296875",' "}]},
    "facebook_positions":["feed","instant_article",
    "instream_video","marketplace"],
    "device_platforms":["mobile","desktop"],
    "publisher_platforms":["facebook","messenger"],
    "messenger_positions":["messenger_home"]}')

# remove spaces
query <- str_replace_all(query, "[\\n]", "") |>
  str_replace_all(" ", "")

# extract
query_val <- url(query) %>% fromJSON
query_val$data
```

	daily_outcomes_curve	estimate_dau	estimate_mau_lower_bound
1	0, 0, 0, 0	1347431	1900000
	estimate_mau_upper_bound	estimate_ready	
1	2200000	TRUE	

7 Exercises

1. Extract estimates for the number of men in Ontario.
2. Repeat 1. but for women, and combine the results of both into the one data frame with a gender column.
3. Using the **GET** function, extract the types of behaviors that are available to extract.
4. Based on the information from part 3., extract estimates of the number of small business owners in Canada.
5. Extract information on the number of small business owners who have also recently moved.